

למידה עמוקה - Multi-layer Perceptrons

למידה עמוקה היא נושא בתוך למידת מכונה. ה-perceptron מוגבל בגלל שהוא מפריד לינארית. למשל - הוא לא יודע לפתור את בעיית הזוגיות (האם מספר הביטים הדולקים זוגי) אפילו בהינתן אינסוף data ואפילו עבור $n = 2$ (כלומר XOR) - למרות שזו בעיה מאוד פשוטה אלגוריתמית.

אבל זה שפרספטרון לא יכול לפתור את הבעיה, לא אומר שאי אפשר ללמוד מנתונים. פרספטרון הוא מודל לינארי - הוא יכול לפתור רק בעיות לינאריות. מה שהוציא את התחום מתוך "החורף הגרעיני" של הפרספטרונים הוא רעיון שבמקום להאכיל את הפרספטרון ישר בנתונים אפשר לעשות עליהם מניפולציה כדי שיהפוך להיות פריד לינארית.

למשל - עבור בעיית XOR, אם יש לנו x_1, x_2 אפשר להפוך אותם ל $x_1 + x_2$ ו $x_1 \cdot x_2$, מה שהופך את הXOR לפריד לינארית. אבל זו בעיה פשוטה - במימד יותר גבוה:

- יותר קשה לראות את זה.

- איזה טרנספורמציות מותר לעשות?

- איזה אסור לעשות?

- איך למצוא אותם?

אנחנו יכולים להשתמש בסוג פשוט של טרנספורמציות, שמורכבות רכיב לינארי - פרספטרון - ורכיב לא-לינארי.

האתגר: במקום להמציא משקלים לטרנספורמציות האלה בעצמנו, איך נוכל ללמוד אותם מתוך נתונים?

הדבר זה נותן פונקציה, עם מבנה מאוד מסויים - אבל מסתבר שאם הגודל של הרשת הזאת מספיק גדול, היא יכולה ללמוד כל פונקציה.

ארכיטקטורה

תזכורת: workflow הסטנדרטי של machine learning מורכב משני שלבים:

1. שלב למידה

2. שלב הסקה

נניח קודם שמישהו למד כבר את המודל ונראה איך מבצעים את ההסקה. אחרי זה נראה איך הלמידה מתבצע.

• מתחילים מיחידה בודדת שמתנהגת כמו פרספטרון - כלומר ווקטור קלט x ווקטור משקלות w .

• הפרספטרון מחשב $w^T x = \sum_i w_i x_i$ ומפעיל עליו פונקציית הפרדה - למשל $z = \text{sign}(w^T x)$ או $z = \sigma(w^T x)$.

שאלה: למה צריך אי-לינאריות?

תשובה: בלי זה, כל שכבה בעצם הייתה הפעלת מטריצה - ואז היה אפשר באמצעות כפל מטריצות לכנס אותן למטריצה בודדת והכל יהיה שקול לשכבה אחת בודדת.

אבל: יכול להיות שתהליך הלמידה יכנס אותנו לפתרונות אחרים מאשר שכבה בודדת (למרות שמחלקת הפתרונות אותו הדבר)

• אנחנו נשתמש בכמה כאלה בכל שכבה - כלומר נחשב $z_1 = \sigma(w_1^T x)$, $z_2 = \sigma(w_2^T x)$. זה נותן

$$z = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \end{bmatrix} \text{ לנו ווקטור}$$

• את הווקטור הזה אנחנו מכניסים לשכבה הבאה - עם משקלות משלה (נסמן u) - ונחשב $\sigma(u^T z)$

• אפשר לעשות כמה שכבות כאלה, ובסוף לעשות למשל softmax.

איך בוחרים איזה פונקציה של אי-לינאריות לשים?

• פונקציית sign היא לא טובה, שכן היא לא חלקה ולא גזירה - קשה לפעפעה דרכה מידע איך הפרמטרים משפיעים על הפלט.

• סיגמואיד ($\sigma(x) = \frac{1}{1+e^{-x}}$) הוא גזיר, אבל הוא מעביר מידע אחורה רק בתחום הדינאמי (קרוב לאפס, איפה שהוא מתחלף) וגם קשה יותר לחשב אותו (אקספוננט דורש פנייה לטבלאות - יותר קשה לממש אותו בחומרה)

• יש פונקציית אקטיבציה פשוטה בשם ReLU (Rectified Linear Unit) - שהיא בעצם $\max(0, x)$. היא מאוד מהירה לחישוב, אבל עלולה להתפוצץ עבור ערכים גבוהים.

- יש גם שיפור כמו Leaky ReLU, שהוא $\max(0.1x, x)$.

למידה

אנחנו צריכים להגדיר loss function ולהביא ולהביא אותו למינימום באמצעות Stochastic Gradient Descent.

הערה: בשלב הזה לא נראה איך לומדים את הצורה של המודל אלא רק את המשקלים. בעבר היה מאוד חשוב לבחור ארכיטקטורה, אבל כיום, עם שיטות מודרניות, כל עוד הארכיטקטורה עשירה מספיק, הביצועים של הרשת לא רגישים יותר מדי למבנה שלה.

מה צריך ללמוד?

- את המשקלות w של כל השכבות.
- את הביאס b . למרות שאפשר במקום זה לשים בכל שכבה רכיב אחד בקלט שערכו קבוע ל1.
- איזה loss נבחר? loss מאוד טבעי הוא לספור את מספר הטעויות (loss 0-1). אבל:
 - זה מידע מאוד גס - עבר או נכשל. רוצים feedback יותר עדין.
 - זו לא פונקציה רציפה, לא גזירה, ובמקומות בהן היא גזירה - הנגזרת היא אפס. במקום היחיד המעניין הנגזרת היא אפס.
- פונקציית loss יותר טובה היא hinge-loss, שבה משלמים מחיר גם אם צדקנו - מחיר שקטן ככל שאנחנו "מתרחקים" מנקודת ההפרדה (אבל כמובן אם טועים משלמים יותר)

Back-propagation of the gradient

יש המון פרמטרים (במקרים של מודל שפה - מיליארדים גם עבור המודלים הקטנים) ולכן חשוב למצוא שיטה יעילה לחישוב הגרדיאנט.

- הרעיון: בגלל שהמודל בנוי בשכבות, אפשר לגזור לפי כלל השרשרת.
- אפשר לשמור חישובים בין שכבות בשביל לחסוך כפילויות.

יש לנו $\text{loss}_W(f(x_i), y_i)$. אנחנו רוצים לחשב את הגרדיאנט. למצוא את המשקלות של השכבה האחרונה u זה מאוד פשוט. בשביל לגזור את W משתמשים בכלל השרשרת:

$$\frac{d}{dw} \text{loss}_W(f(x_i), y_i) = \frac{d \text{loss}}{df_w} \frac{df_w}{d\sigma(w^T x)} \frac{d\sigma(w^T x)}{dw}$$

למשל - אם פונקציית האי-לינאריות שלנו היא סיגמואיד, אז:

$$\frac{d \text{loss}}{dw} = \frac{d \text{loss}}{df_w} \frac{df_w}{d\sigma} \frac{d\sigma}{dz} \frac{dz}{dw} = (-y) \sum_j u_j \sigma(a_j (1 - \sigma(a_j))) x$$

- האלגוריתם: עושים forward pass, ושומרים את חישובי הביניים.
- עושים backward pass. מסתכלים על הloss, מפעפעים אותו בחזרה, ומתקנים בכל שלב את המשקלות.

ארכיטקטורת למידה עמוקה - Convolution Neural Networks

המחלקה של רשתות ניורונים הבנויות משכבות מאוד אקספרסיבית, ובהינתן מספיק ניורונים יכולה לייצג בפועל כל פונקציה. אבל יש לה בעיות:

- המורכבות מאוד גדולה. למשל - נגיד תמונה של 1000 על 1000 פיקסלים, עם ערכי RGB לכל פיקסל, ווקטור הקלט הוא בגודל $6,000,000 = 3 \times 1000 \times 1000$. אז לכל ניורון בשכבה השנייה נצטרך 6 מיליון משקלות (אחד לכל ערך בקלט) - וזה רק לניורון בודד.
- הנגזרת של הסיגמואיד די מוגבלת בגודל שלה (בין 0 ל-1). כלל השרשרת הוא מכפלה של הרבה נגזרות, מה שמקטין מאוד את המספרים - ואז קשה ללמוד כי עד שמפעפעם לשכבה הראשונה השינויים מאוד קטנים.
- אם מנרמלים שיהיה יותר מאחד, זה מתפוצץ.
- כל משקלות בשכבה הראשונה יכולה להשפיע על הפלט בהרבה מאוד דרכים - ואז באופן ממוצע, בגלל שיש הרבה נתיבים לעדכן אותן, המשקלות נעשות מאוד קרובות לאפס (כי יש עדכונים חיוביים ושליילים דרך נתיבים שונים)
- מאתחלים במשהו רנדומי - ואז כל ההמשך נהיה רנדומי.
- יש הרבה רשתות שקולות:

- למשל - אם פשוט המשקלות של צמתים באותה שכבה פנימית מתחלפים אחד עם השני.

- פונקציית הסיגמואיד היא אנטי-סימטרית - $\sigma(-x) = 1 - \sigma(x)$ - לכן אפשר להפכיל משקלות ב-1 גם בכניסה וגם ביציאה מצומת ולקבל רשת שקולה.

- לפונקציית הloss יש הרבה מימדים - ולכן הרבה דרכים למצוא מינימום - ובגלל כל השכפולים יש גם הרבה מינימומים לוקאליים גרועים.

לכן הרשתות האלה בהתחלה, בשנות ה-80, היו מוגבלות ל-3 או 4 שכבות - כי אם יש יותר שכבות נעשה מאוד קשה לאמן. גם לא הייתה הבנה טובה של overfitting והגדלים של ה-datasets היו מאוד קטנים - בעיקר בהשוואה למספר הפרמטרים העצום. היינו רוצים להקטין את מרחב הפרמטרים - אבל בלי להקטין את האקספרסיביות. כלומר - "לזרוק" רק פונקציות אפשריות שלא מעניינות אותנו.

בעיות של MLP

- כל ניורון בשכבה הראשונה עושה ממוצע משוקלל על ערכי הקלט. הרשת לא מודעת באמת לסדר - אם מחליפים פיקסלים (באופן עקבי) הרשת תלמד משהו מאוד דומה.
- זוהי אינווריאנטיות שאנחנו לא רוצים - הרשת מנסה לפתור בעיה הרבה יותר קשה מסתם ללמוד תמונות.

- הרשת לא אינווריאנטית להזזות. אם קצת מזיזים את התמונה (לא באופן עקבי!) המודל רואה משהו אחר לגמרי, כי כל פיקסל מראה משהו אחר ממה שהוא למד.
- זוהי אינווריאנטיות שכן היינו רוצים שתהיה.

הפתרון - Convolution Neural Network

- הרעיון הבסיסי דומה, אבל המבנה של המשקלות שונה:
- במקום שכל שכבה תסתכל על כל השכבה הקודמת, כל ניורון בשכבה מסתכל רק על אזור מוגבל.
 - ווקטור המשקלות - נקרא filter - זהה לכל הניורונים בשכבה.
 - אינטואיטיבית - אם ווקטור המשקלות הזה יודע לתפוס איזו תבנית בתמונה, זה לא משנה איפה התבנית תהיה - כי מפעילים אותן משקלות על כל הקלט.
 - כל שכבה קצת יותר קטנה מהשכבה הקודמת (כי הקונבולוציה לא חורגת מהקצוות)
 - אפשר לשים כמה פילטרים בכל שכבה - כל אחד הוא ווקטור משקלות משל עצמו שפועל שוב ושוב על השכבה הקודמת.
 - על כל שכבה כזאת מפעילים אי-ליניאריות (שוב - איבר איבר) ולהפעיל את השכבה הבאה.
 - נשים לב שבשיטה הזאת התמונה שלנו היא תלת-מימדית - אם מחשיבים את הצבע (RGB) בתור מימד. בשכבות הבאות מימד העומק מייצג פיצ'רים. כל צומת מפעיל קונבולוציה על כל העומק.
 - אפשר באיזהו שלב לעבור לfully connected. אבל עושים את זה רק בשכבות האחרונות, כי זה מאוד בזבזני במספר הפרמטרים.
 - בנוסף לשכבות האי-ליניאריות הרגילות, יש גם שכבות של MAX-Pooling - לוקחים את המקסימום מתוך איזור (לכל פיצ'ר בנפרד)
 - ככל שמתקדמים בשכבות, כל יחידה מושפעת מאזור יותר גדול מהקלט - ולכן כנראה עובדות לא על פיקסלים גולמיים אלא על פיצ'רים שהשכבות הקודמות למדו להוציא.

חומר למחשבה

- אינווריאנטיות:
 - אין אינווריאנטיות לסיבוב
 - * לפעמים רוצים, לפעמים לא
 - * אפשר לעשות מניפולציה לdata.
 - אין אינווריאנטיות להגדלה
- יש ארכיטקטורות אחרות בשביל להשיג אינווריאנטיות לדברים אחרים.
- איך בוחרים את גודל הקונבולוציות?
 - פילטרים יותר גדולים יכולים למצוא פיצ'רים יותר גדולים ומסובכים.
 - מצד שני, משלמים על זה בפרמטרים ובהכללה.